

Séance 2 : Apache Kafka : Ingestion distribuée

Architecture interne, producers, consumers, garanties de livraison et monitoring

Dr. Khalil Haddaoui

- ① Rappels et rôle de Kafka dans un pipeline temps réel
- ② Architecture interne de Kafka
- ③ Modèle publication / souscription distribué
- ④ Producers avancés
- ⑤ Consumers avancés
- ⑥ Garanties de livraison : at-most-once, at-least-once, exactly-once
- ⑦ Monitoring et performance : lag, débit, latence
- ⑧ Synthèse et préparation du TP

Intuition

Cette séance répond à une question simple : **Comment construire une ingestion d'événements fiable, scalable et observable ?**

Objectifs pédagogiques de la séance

À la fin de cette séance, vous serez capables de :

- expliquer l'**architecture interne** de Kafka (brokers, topics, partitions, réplicas, segments) ;
- décrire précisément le **modèle pub/sub distribué** et le rôle du partitionnement ;
- configurer un **producer fiable** (acks, batch, compression) et comprendre ses compromis ;
- configurer un **consumer robuste** (gestion des offsets, rebalancing, isolation) ;
- analyser les **garanties de livraison** et identifier où se produisent pertes / doublons ;
- interpréter le **Kafka lag**, le débit et la latence pour diagnostiquer un pipeline.

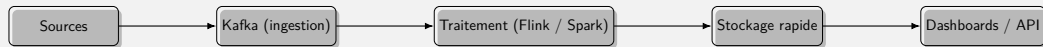
Positionnement dans la formation

Séance 1 : concepts temps réel et introduction à Kafka. **Séance 2** : plonger dans le fonctionnement interne de Kafka pour construire une *ingestion robuste*.

Intuition

Ces notions correspondent aux compétences attendues d'un **Data Engineer temps réel** en entreprise.

Kafka dans le pipeline temps réel (rappel)



Rôle de Kafka

- **Tampon** entre les producteurs d'événements et les systèmes de traitement.
- **Découplage** dans le temps : les consumers peuvent arriver plus tard.
- **Log distribué** persistant, relisable, partitionné.

Exemple concret

Pendant un **Black Friday**, le trafic explose. Les systèmes de traitement ne suivent pas le rythme en temps réel, mais Kafka stocke tout sans perte et permet de rattraper ensuite.

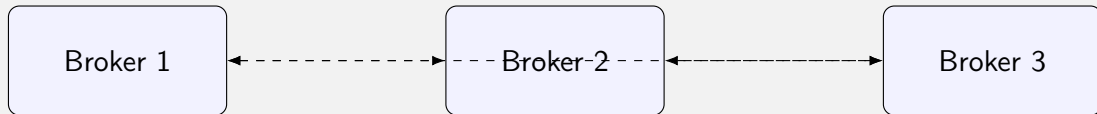
Mini-rappel : notions clés de la séance 1

- Un **événement** = (type, timestamp, key, value).
- Désordre naturel entre **Event Time** et **Processing Time**.
- Besoin de **garder une trace ordonnée** des événements par clé $\Rightarrow$ log.
- Kafka : déjà vu comme :
 - un ensemble de **topics** (flux logiques),
 - chaque topic $\rightarrow$ plusieurs **partitions**,
 - chaque partition $\rightarrow$ suite immuable de messages indexés par **offset**.

Séance 2

On ouvre maintenant la “boîte noire” Kafka : **brokers, réplicas, segments, transactions, lag, etc.**

Cluster Kafka

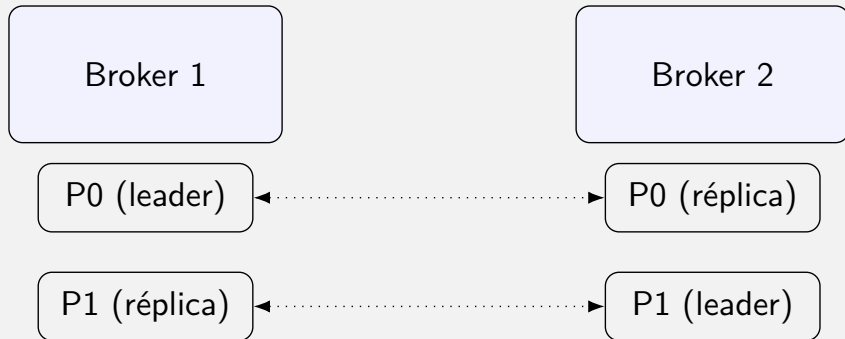


- Un **cluster** Kafka = plusieurs **brokers**.
- Les **topics** sont répartis en **partitions** sur les brokers.
- Chaque partition a un **leader** et éventuellement des **réplicas**.

Idée clé

Tolérance aux pannes = **réplication** des partitions sur plusieurs brokers.

Topic "paiements"



- **P0** et **P1** sont des partitions d'un même topic.
- Pour chaque partition :
 - un **leader** gère les lectures / écritures,
 - un ou plusieurs **réplicas** assurent la redondance.

Définition

Les réplicas à **jour** (synchronisés avec le leader) forment l'ensemble **ISR**. En cas de panne du leader, Kafka choisit le nouveau leader **dans l'ISR**.

Exemple

Partition P0 :

- Leader sur Broker 1,
- Réplicas sur Brokers 2 et 3.

Si Broker 1 tombe :

- Kafka promeut un réplica **ISR** (par ex. Broker 2) comme nouveau leader,
- on évite de perdre les derniers messages confirmés.

Attention

Un réplica **hors ISR** ne doit jamais devenir leader, sinon les derniers messages peuvent être perdus.

Intuition

L'ISR, c'est la **liste des copies fiables**. Si le leader tombe, Kafka choisit un nouveau leader **dans l'ISR** pour garantir que les derniers messages ne sont pas perdus.

Exemple

Topic transactions_bancaires, partition P0 :

- Leader sur Broker 1.
- Réplicas sur Brokers 2 et 3.
- $ISR = \{\text{Broker 1, Broker 2}\}$ (Broker 3 est en retard).

Si Broker 1 tombe :

- Kafka promeut Broker 2 (dans l'ISR) comme leader.
- Broker 3 ne peut pas devenir leader tant qu'il n'est pas à jour.

Un réplica hors ISR ne doit jamais devenir leader, sinon les **derniers événements confirmés** peuvent être perdus.

Leader, followers et tolérance aux pannes

- Les **producers** écrivent **sur le leader** d'une partition.
- Les **followers** répliquent les données du leader.
- En cas de panne du leader :
 - un follower **in-sync** devient le nouveau leader ;
 - transparence pour les applications (rebalancing côté clients).

Scénario

- Partition P0 : leader sur Broker 1, réplicas sur Broker 2 et 3.
- Broker 1 tombe :
 - Kafka élit un nouveau leader (par ex. Broker 2),
 - les producers / consumers redirigent automatiquement leurs flux.

Point d'attention

Si les réplicas ne sont pas à jour (hors ISR), on peut perdre **les derniers messages**.

- Chaque **partition** est stockée comme une suite de **segments de fichiers**.
- Un segment contient :
 - des messages (log),
 - des **index** pour retrouver rapidement un offset.
- Quand un segment est trop gros (taille ou temps) :
 - Kafka **roule** vers un nouveau segment (rollover).

Pourquoi c'est important ?

- Permet d'effacer ou de compacter les vieux segments sans toucher aux récents.
- Donne une **structure interne** pour appliquer les politiques de rétention.

Rétention classique

- **Time-based** : garder les messages pendant X jours.
- **Size-based** : garder jusqu'à Y Go par partition.

Log-compaction

- Par clé, Kafka ne garde que la **dernière valeur** (et éventuellement quelques anciennes).
- Adapté pour des topics de “**table de référence**” :
 - ex : topic user-profile avec la dernière version du profil utilisateur.

Choix pédagogique

En pratique, on combine souvent :

- un topic **log complet** (audit),
- un topic **compacté** (vue actuelle par clé).

Intuition

- **Rétention** : on garde l'historique complet pendant un temps limité, puis on **supprime tout**.
- **Compaction** : on garde **une version par clé** (la plus récente), même longtemps.

Exemples

- Topic clicks : rétention 7 jours, pas de compaction. On veut analyser les flux bruts récents.
- Topic user-profile : compaction activée. On veut toujours la dernière version par utilisateur.

Attention

Ne pas activer la compaction sur des topics **d'événements métier** (log d'actions) si l'historique complet est nécessaire pour les analyses.

- Les **producers** n'ont pas besoin de connaître les consumers.
- Les **consumers** n'ont pas besoin de connaître les producers.
- Kafka agit comme une **zone tampon durable**.

Conséquences pratiques

- On peut ajouter un nouveau consumer (ex : “modèle de fraude v2”) sans changer les producers.
- On peut **rejouer** l'historique pour tester un nouveau modèle.

Idée clé

Kafka permet de “**geler**” les flux d'événements dans un log, puis de les **relire à volonté**.

Partitionnement : pourquoi, comment ?

- Kafka **distribue** les messages d'un topic sur plusieurs partitions.
- Objectifs :
 - **scalabilité** (plus de débit),
 - **parallélisme** de traitement,
 - **ordre garanti par clé**.

Stratégies de partitionnement

- **Hash de clé** : $\text{partition} = \text{hash}(\text{key}) \bmod N$.
- **Round-robin** : sans clé, on répartit de manière uniforme.
- **Custom partitioner** : logique métier spécifique.

Point crucial

L'ordre est **garanti** uniquement à l'intérieur d'une **même partition**. Si une même clé est envoyée sur plusieurs partitions, on perd cet ordre.

Contexte : topic "clicks" pour un site e-commerce.

- Clé possible 1 : `session_id`.
- Clé possible 2 : `user_id`.
- Clé possible 3 : `page_url`.

Questions

- ❶ Si vous voulez reconstruire le parcours d'une **session**, quelle clé choisir ?
- ❷ Si vous voulez analyser le comportement **global** par page, quelle clé est pertinente ?
- ❸ Quel risque si vous utilisez `null` comme clé (round-robin) pour les 3 cas ?

Objectif pédagogique

Relier le **choix de la clé** à la **vue métier** que l'on veut reconstruire.

- Prendre un **événement métier** (structuré) et le sérialiser (JSON, Avro, Protobuf...).
- Choisir :
 - le **topic**,
 - la **clé** (pour le partitionnement),
 - éventuellement la partition.
- Configurer le **niveau de fiabilité** :
 - `acks`,
 - `retries`, `linger.ms`, `batch.size`, `compression.type`, etc.

Vision

Un producer est un **adaptateur** entre le monde applicatif et le **log distribué** Kafka.

acks=0

- Le producer n'attend **aucune** confirmation.
- **Très rapide**, mais on ne sait pas si le message est arrivé.

acks=1

- Confirmation du **leader** uniquement.
- Bon compromis débit / fiabilité.

acks=all

- Attente de la réplication sur **tous les ISR**.
- Plus **fiable**, mais plus de **latence**.

Question métier

Dans votre contexte (finance, IoT, web), **où pouvez-vous perdre un message ? où ne pouvez-vous surtout pas en perdre ?**

Batching

- `batch.size` : taille max d'un batch envoyé d'un coup.
- `linger.ms` : temps d'attente pour remplir le batch.
- Plus les batches sont gros, plus le **débit** est élevé (mais latence plus grande).

Compression

- `compression.type` : none, gzip, snappy, lz4, zstd.
- Réduit le volume réseau et disque, mais consomme de la CPU.

Règle d'or

On ajuste ces paramètres en fonction de :

- la **tolérance à la latence**,
- le **débit** souhaité,
- la **charge CPU** acceptable.

Producer idempotent

- Permet d'éviter les **doublons** en cas de retries.
- Kafka assigne un **producer ID (PID)** et suit les séquences de messages.

Transactions Kafka

- Permettent des **écritures atomiques** sur plusieurs partitions / topics.
- Utiles pour :
 - les scénarios **exactly-once** avec un moteur de streaming,
 - les opérations “lire puis écrire” (read-process-write).

Complexité

Les transactions améliorent la sémantique (exactly-once), mais ajoutent de la **complexité** de configuration et de monitoring.

Intuition

- **Idempotence** : réémettre le même message plusieurs fois produit le **même effet final**.
- **Transaction** : écrire sur plusieurs partitions/topics de manière **tout ou rien**.

Exemple

Pipeline :

- Lecture d'un événement de paiement.
- Écriture dans un topic audit et un topic paiements-traités.

Sans transaction : un seul des deux topics peut être écrit → incohérence. Avec transaction : soit les deux sont écrits, soit aucun.

Attention

Les transactions coûtent en latence et complexité. À réserver aux cas où **consistance forte** et **exactly-once** sont réellement nécessaires.

- Lire les messages séquentiellement depuis une ou plusieurs **partitions**.
- Les **désérialiser** et appliquer une logique métier.
- **Gérer les offsets** : que s'est-il passé si je tombe en panne ?

Décision clé

À quel moment **committe-t-on** l'offset ?

- avant traitement $\Rightarrow$ risque de perte en cas de crash,
- après traitement $\Rightarrow$ risque de **rejouer deux fois**.

Auto-commit

- Kafka commite périodiquement l'offset pour vous.
- Simple, mais **moins de contrôle** :
 - on peut perdre des messages ou en rejouer selon le timing.

Commit manuel

- L'application décide quand commiter :
 - après traitement réussi,
 - éventuellement par **batch**.
- Permet d'implémenter un **at-least-once** robuste (au prix de duplications possibles).

Bonne pratique

Toujours réfléchir : “**Si mon consumer tombe maintenant, que va-t-il se passer ?**”

- Un **consumer group** = plusieurs instances qui consomment un même topic.
- Kafka répartit les **partitions** entre les consumers du groupe.
- Si un consumer rejoint ou quitte le groupe :
 - Kafka déclenche un **rebalancing**,
 - les partitions sont redistribuées.

Impact du rebalancing

Pendant un rebalancing :

- certains consumers arrêtent temporairement de consommer,
- la latence peut **augmenter**,
- le **lag** peut augmenter puis redescendre.

Intuition

Le rebalancing, c'est comme **réorganiser une équipe** :

- on arrête un peu la production,
- on redistribue les tâches,
- puis on reprend.

Exemple

- 3 consumers pour un topic à 6 partitions $\Rightarrow$ 2 partitions par consumer.
- Un 4^e consumer arrive $\Rightarrow$ Kafka redistribue 6 partitions sur 4 consumers.

Pendant cette redistribution, certains messages ne sont plus consommés $\Rightarrow$ **lag temporaire**.

Attention

Un rebalancing trop fréquent (consumers instables, timeouts trop courts) peut rendre le pipeline **très instable en latence**.

`read_uncommitted`

- Le consumer voit **tous** les messages, même ceux issus de transactions non commitées.
- Potentiellement plus rapide, mais risque de lire des données rollbackées.

`read_committed`

- Le consumer ne voit que les messages **de transactions commitées**.
- Cohérent avec les scénarios **exactly-once**.

Lien avec la suite

Ces notions sont essentielles pour comprendre la **sémantique de livraison** : at-most-once, at-least-once, exactly-once.

- On **committe l'offset avant** de traiter le message.
- Si l'application tombe après le commit, le message est **perdu**.

Utilisation possible

- Logs non critiques.
- Métriques approximatives, où une petite perte est acceptable.

Risque

Pas acceptable pour :

- transactions financières,
- événements réglementaires,
- comptage de type “facturation”.

At-least-once : au moins une fois

- On **committe l'offset après** le traitement.
- Si l'application tombe après le traitement mais avant le commit :
 - le message sera **retraité** à la reprise.

Conséquence

- Pas de **perte** de message.
- Mais possible **duplication** du traitement.

Approche standard

On combine souvent :

- **At-least-once** + **logique idempotente** côté traitement.

- Objectif : **ni perte, ni duplication** du point de vue du résultat final.
- En pratique : combinaison de :
 - **transactions Kafka**,
 - **state managé** par le moteur de streaming (ex : Flink),
 - **sink transactionnel** ou idempotent.

Important

“Exactly-once” signifie “exactement une fois **au niveau du résultat**”, pas forcément “exactement une fois au niveau des **appels internes**”.

Intuition

On peut **rejouer** plusieurs fois un même message, mais si le système est bien conçu, **le résultat final** est comme s'il avait été traité une seule fois.

Exemple

Un paiement est traité deux fois à cause d'un crash/restart :

- Avec une logique idempotente côté base de données (clé unique), la 2^e écriture ne change pas le résultat.
- Côté utilisateur, la transaction n'apparaît qu'une fois.

Exactly-once dépend :

- du producer (transactions),
- du moteur de streaming (gestion de state),
- du sink (idempotent / transactionnel).

Ce n'est jamais "magique".

Où se produisent pertes et doublons ?

- **Entre producer et broker :**
 - pertes possibles si `acks=0` ou configuration réseau fragile.
 - doublons possibles si retries sans idempotence.
- **Entre broker et consumer :**
 - pertes possibles si on commite avant traitement (at-most-once).
 - doublons possibles si on commite après traitement (at-least-once).
- **Entre consumer et sink :**
 - dépend de la sémantique du sink (DB, cache, etc.).

Question à se poser

Pour chaque “maillon” de la chaîne, **quelle sémantique de livraison** est réellement obtenue ?

Définition

Le **lag** d'un consumer group sur une partition = **(dernier offset produit) - (dernier offset commité)**.

- Lag **proche de 0** :
 - consumer “à l'heure”.
- Lag **qui augmente** :
 - consumer **en retard**, risque de latence importante.

Interprétation pratique

- Si le lag explose : **débit des producers > capacité des consumers**.
- Actions possibles :
 - augmenter le **parallélisme** (plus de partitions / consumers),
 - optimiser le code du consumer,
 - réduire la complexité des traitements.

Intuition

Le lag mesure le **retard accumulé** du consumer par rapport au producteur. C'est l'un des indicateurs les plus importants de la **santé** d'un pipeline Kafka.

Exemple

- Lag = 0 $\Rightarrow$ les événements sont traités quasi en temps réel.
- Lag = 1 000 000 $\Rightarrow$ le consumer a **un million de messages de retard**.

Sur un flux critique (paiements, alertes), cela peut être inacceptable.

Attention

Un lag qui augmente en continu signifie :

- soit le **débit producer** a augmenté,
- soit le consumer est devenu **plus lent** (code, I/O, rebalancing, etc.).

Il faut agir avant que la latence end-to-end ne devienne trop grande.

- **Throughput** :
 - messages/s, octets/s.
 - mesuré côté producers, brokers et consumers.
- **Latence end-to-end** :
 - temps entre **Event Time** et arrivée dans le sink final,
 - dépend de Kafka **ET** du moteur de streaming.

Compromis à gérer

- Plus de **sécurité** (acks, transactions) $\Rightarrow$ plus de latence.
- Plus de **débit** $\Rightarrow$ risque de lag et parfois de perte de messages mal configurés.

- **Kafka UI / outils graphiques :**
 - visualiser topics, partitions, lags.
- **JMX + Grafana / Prometheus :**
 - métriques détaillées : débit, latence, taille des files, erreurs.
- **CLI Kafka / kcat :**
 - inspection rapide d'un topic, d'une partition,
 - consommation ponctuelle pour debug.

Message

Un pipeline temps réel **sans monitoring** est un pipeline qui finira par échouer **sans que vous le voyiez venir**.

Nous avons approfondi :

- l'**architecture interne de Kafka** : brokers, partitions, réplicas, ISR, segments, rétention, log-compaction ;
- le **modèle pub/sub distribué** et l'importance du **partitionnement par clé** ;
- les **producers avancés** : acks, batching, compression, idempotence, transactions ;
- les **consumers avancés** : gestion fine des offsets, consumer groups, rebalancing, isolation levels ;
- les **garanties de livraison** : at-most-once, at-least-once, exactly-once, et où se produisent pertes / doublons ;
- le **monitoring** : lag, throughput, latence, et quelques outils pour les suivre.

À retenir

Kafka n'est pas qu'une file de messages : c'est une **colonne vertébrale** de vos systèmes temps réel.

Dans le **TP de la Séance 2**, vous allez :

- créer un **cluster Kafka local** ;
- créer et configurer un **topic** (rétention, partitions) ;
- implémenter un **producer Python fiable** (acks=all, compression, batching) ;
- implémenter un **consumer Python** avec commits manuels ;
- simuler :
 - **perte de messages** (mauvaise config),
 - **duplication volontaire** pour comprendre at-least-once ;
 - **retard d'un consumer** et analyser le **lag**.
- tester le **replay** et visualiser l'évolution des offsets.

Objectif

Passer de la vision **conceptuelle** de Kafka à une pratique **concrète et instrumentée** de l'ingestion distribuée.